Deliverable 4: Advanced Database Concepts — Procedures, Triggers & CTEs

Project: Intelligence Fusion Platform • **Course:** DBMS Mini Project (UE24CS395)
Authors: Saatvik Gupta (PES1UG24CS395) & Satyam Goyal (PES1UG24CS424) • PES University (2026–2027)

1. Stored Procedure & User-Defined Function

```
-- Stored Procedure: Atomic Case Creation & Audit Logging
CREATE OR REPLACE PROCEDURE sp_create_investigation_case(
    p_case_id VARCHAR, p_title VARCHAR, p_description TEXT,
    p_case_type VARCHAR, p_priority VARCHAR, p_officer_id VARCHAR
)
LANGUAGE plpgsql AS $$
BEGIN
    INSERT INTO cases (case_id, title, description, case_type, status, priority, lead_officer_id)
    VALUES (p_case_id, p_title, p_description, p_case_type, 'Open', p_priority, p_officer_id);

    INSERT INTO case_activity (case_id, officer_id, activity_type, title, description)
    VALUES (p_case_id, p_officer_id, 'Procedure Execution', 'Case Registered', 'Initialized via sp_create_investigation_case.');
    COMMIT;
END;
$$;
```

2. Automatic State Transition Audit Trigger

```
-- Database Trigger: Automatically audits status/priority transitions
CREATE OR REPLACE FUNCTION fn_trg_case_audit()
RETURNS TRIGGER AS $$
BEGIN
    IF (OLD.status IS DISTINCT FROM NEW.status) OR (OLD.priority IS DISTINCT FROM NEW.priority) THEN
        INSERT INTO case_activity (case_id, officer_id, activity_type, title, description)
        VALUES (
            NEW.case_id, NEW.lead_officer_id, 'Trigger Audit',
            'State Transition Detected',
            format('Status: %s -> %s | Priority: %s -> %s', OLD.status, NEW.status, OLD.priority, NEW.priority)
        );
    END IF;
    RETURN NEW;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER trg_case_activity_audit
AFTER UPDATE OF status, priority ON cases
FOR EACH ROW EXECUTE FUNCTION fn_trg_case_audit();
```

3. Recursive CTE: Multi-Hop Criminal Associate Traversal

```
-- Recursive CTE: Discovers associate network up to 5 degrees of separation
WITH RECURSIVE suspect_network AS (
    -- Anchor member: Initial suspect
    SELECT person_id, full_name, 0 AS degree, ARRAY[person_id] AS path
    FROM people WHERE person_id = 'PER-001'
    UNION ALL
    -- Recursive member: Linked associates through common organizations
    SELECT p.person_id, p.full_name, sn.degree + 1, sn.path || p.person_id
    FROM suspect_network sn
    JOIN person_organizations po1 ON sn.person_id = po1.person_id
    JOIN person_organizations po2 ON po1.organization_id = po2.organization_id
    JOIN people p ON po2.person_id = p.person_id
    WHERE NOT p.person_id = ANY(sn.path) AND sn.degree < 5
)
SELECT DISTINCT person_id, full_name, MIN(degree) AS network_depth
FROM suspect_network GROUP BY person_id, full_name ORDER BY network_depth;
```